

Presentation 3

Team 1 - Stat 390

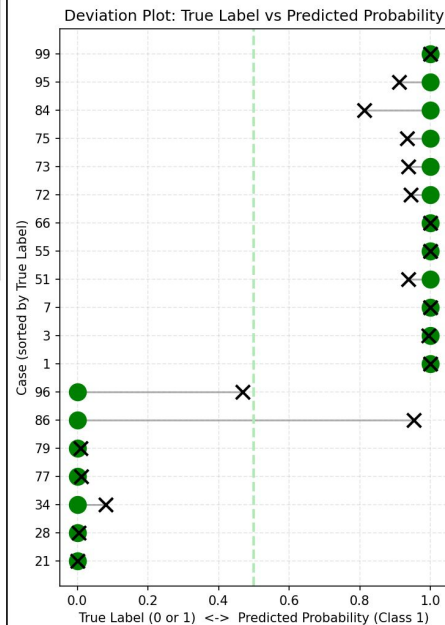
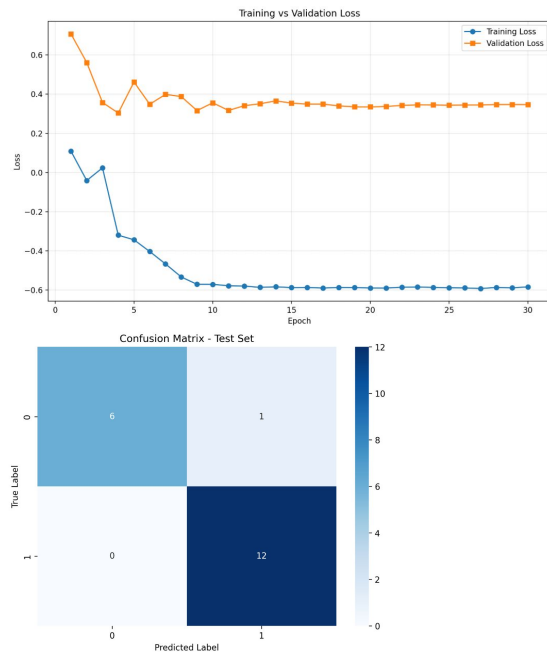
Coding - Mandatory Tasks

- Dropout is commented out of the attention MLP¹
- Gated Attention is implemented²
- Entropy regularization is implemented for attention³
- Splits are changed to `/projects/e32998/STAT390_Krish/Code/Code4_reduce_runtime/data_splits/data_splits.npz`
- Set `early_stopping_minimum_epochs` to 30⁴

1. `Code4_reduce_runtime/models.py`, lines 20 & 26
2. `Code4_reduce_runtime/models.py`, line 32
3. `Code4_reduce_runtime/models.py`, lines 38, 82-87 & `Code4_reduce_runtime/trainer.py` lines 152-153
4. `Code4_reduce_runtime/config.py`, line 50

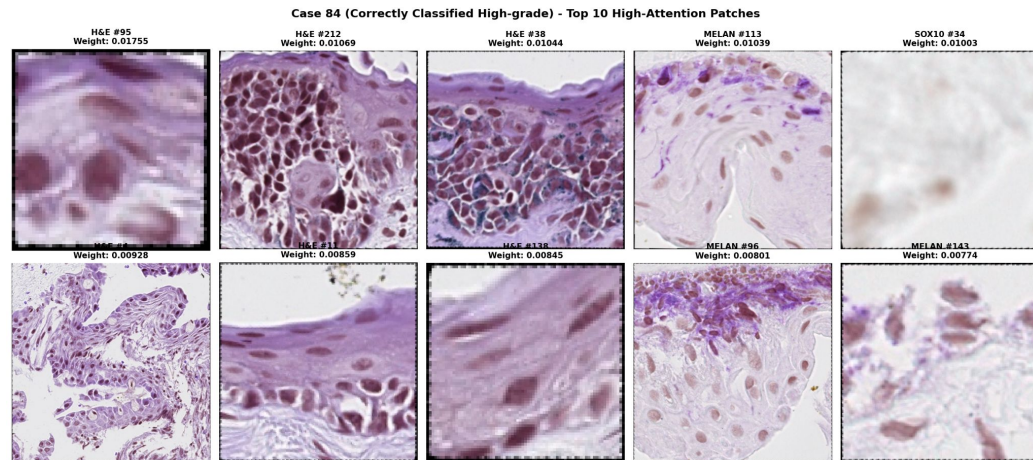
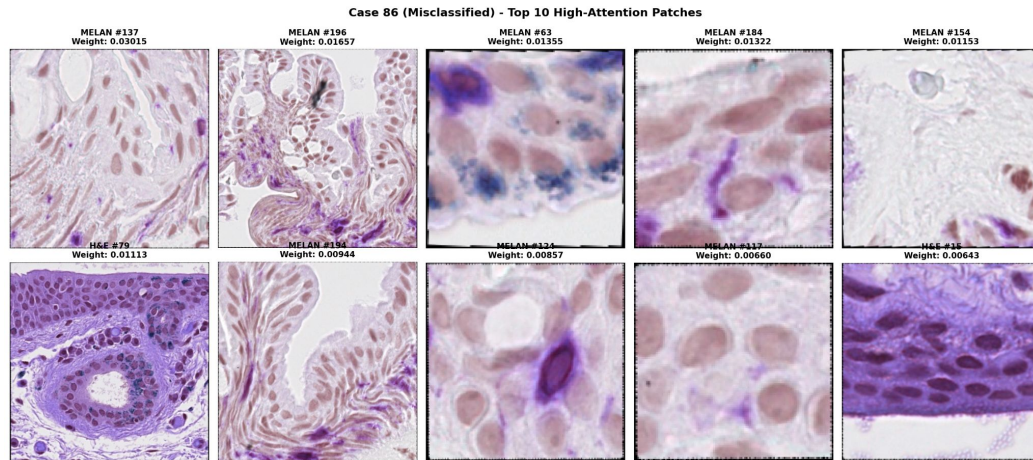
Coding - Best Epoch

- Added `plot_loss_from_log.py` to generate graph of val & training loss vs epoch¹
- Median deviation shows that most predictions are close to the correct target prob
- Mean deviation is much higher than the median showing that the distribution is right skewed
- One high confidence wrong prediction



High attention patches

- Model predicted class 1 for case 86 incorrectly
- Highest attention patches are shown
- Blurry/noisy patches
- Possibly could be fixed by
 - Better patch quality filtering
 - Stain normalization
 - Stronger attention sparsity



Coding - Best Run Visualizations

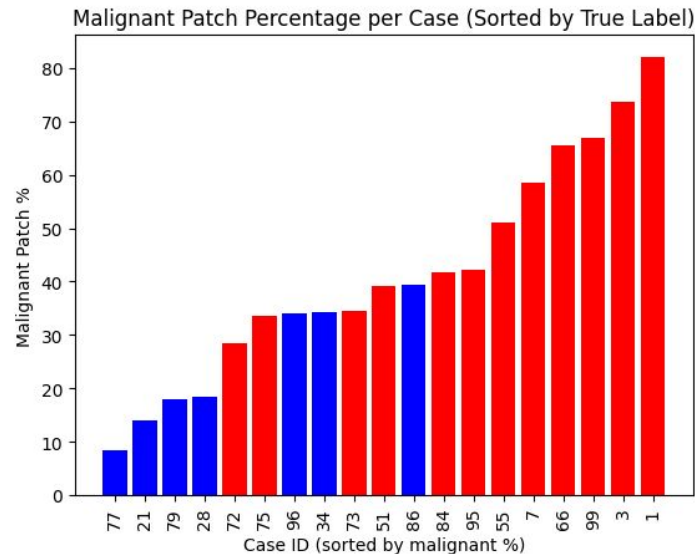
- Implemented `get_patch_level_predictions()` in `models.py`¹
- Created `patch_analysis.py` to analyze the patch level predictions²
- [main.py](#) now outputs a csv file containing information about the patch level-predictions, including percent of patches predicted in both classes³

1. `Code4_reduce_runtime/models.py`, lines 91-122
2. `Code4_reduce_runtime/patch_analysis.py`
3. `Code4_reduce_runtime/main.py`, lines 308-319

Coding - Best Run Visualizations

- Created some visualizations in visualize_patch_analysis.ipynb
- Case 86 was misclassified as malignant - the model sees more of its patches as malignant
- May need to see attention weights in combination with patch_predictions in order to diagnose potential issues

1. Code4_reduce_runtime/visualize_patch_analysis.ipynb



Hyperparameter Tuning

- Goal: improve model performance by systematically tuning 6 key hyperparameters with Optuna Bayesian optimization.
- Hyperparameters tuned:
 - initial_learning_rate
 - weight_decay
 - patch_projector_dropout
 - classifier_dropout
 - class_weight_benign
 - entropy_coefficient
- Optimization strategy:
 - Optuna TPESampler and MedianPruner
 - Hyperparameters selected based on minimum validation loss
- Edited config.py, models.py, trainer.py, optuna_training.py to ensure compatibility across scripts with different dropout variables and all necessary hyperparameters properly specified.

Hyperparameter Tuning - Optuna Run 1

- Run configuration: 10 trials, 30 epochs per trial, 6 epoch pruning warm up, no early stopping.
- Search strategy: explore a wider hyperparameter space (same starting point as hyperparameter tuning for Presentation 2).
- 3 trials pruned (4, 5, 9).
- 7 trials completed. Epoch of best validation loss:
 - Trial 0 → before epoch 5
 - Trial 1 → epoch 10
 - Trial 2 → before epoch 10
 - Trial 3 → before epoch 5
 - Trial 6 → ~epoch 25
 - Trial 7 → before epoch 5
 - Trial 8 → before epoch 10
- Best trial: Trial 1
 - Best validation loss: 0.1914
 - Best validation accuracy: 0.8947

Hyperparameter	Range	Best value
initial_learning_rate	(1e-4 5e-4)	2.63e-04
weight_decay	(5e-5 5e-4)	2.55e-04
patch_projector_dropout	(0.0 0.6)	0.0124
classifier_dropout	(0.0 0.6)	0.5819
class_weight_benign	(1.5 3.5)	3.1649
entropy_coefficient	(1e-5 1e-1)	7.07e-05

Hyperparameter Tuning - Run 1 Results

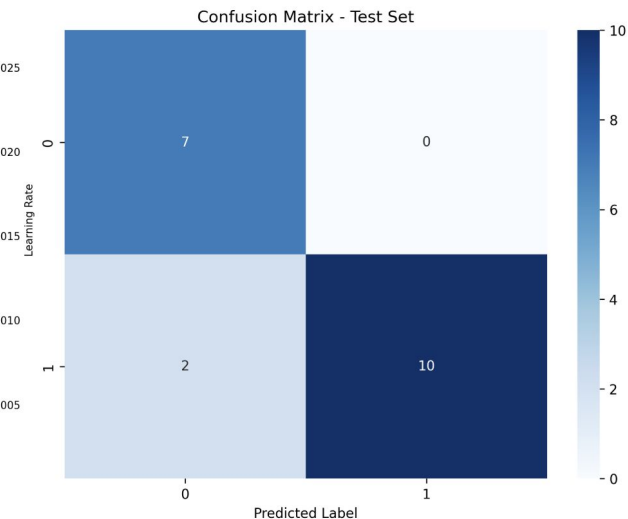
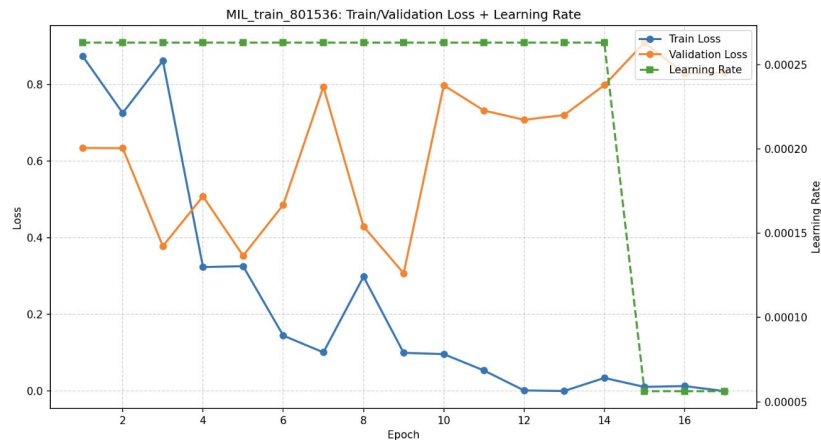
- After tuning, the best hyperparameters were inserted into the training configuration and the model was trained fully.
- Training loss converges quickly, validation loss fluctuates in later epochs (best val loss in epoch 9).
 - Model is memorizing the data very quickly. Parameters aren't updating: gradient step $\rightarrow$ 0 as training loss $\rightarrow$ 0.
- Test accuracy decreased compared to baseline with previously tuned hyperparameters and implemented mandatory changes.
 - High classifier_dropout (0.58) likely weakened the classifier, while small patch_projector_dropout (0.012) allowed the model to memorize training patches with low regularization.
 - The class_weight_benign value (3.1649) was higher than the original tuned hyperparameters, causing the model to be more conservative about predicting high-grade cases, hence why we see 2 false negatives.

```
29 TRAINING_CONFIG = {
30     'epochs': 30, # Increased since we have early stopping
31     'batch_size': 1, # MIL typically uses batch_size=1
32     'learning_rate': 2.63e-04, # Tuned value
33     'weight_decay': 2.55e-04, # Tuned value
34     'patch_projector_dropout': 0.0124, # Tuned value
35     'classifier_dropout': 0.5819, # Tuned value
36     'class_weights': [3.1649, 1.0], # Tuned benign class weight
37     "entropy_weight": 7.07e-05, # Tuned value
```

Test Results:

Test Loss: 0.5672

Test Acc: 0.8947



Hyperparameter Tuning - Optuna Run 2

- Run configuration: 10 trials, 15 epochs per trial, 6 epoch pruning warmup, no early stopping.
 - Use less epochs as 6/7 completed trials in Run 1 achieved best val loss before ~10 epochs.
- Narrowed the hyperparameter search ranges → would have been better to broaden them.
 - Narrowing the search space for several hyperparameters may have limited Optuna's ability to explore better configurations.
 - In particular, `initial_learning_rate`, `weight_decay`, `entropy_coefficient` were already narrow in Run 1 – restricting them further reduced exploration capability.
 - For the `classifier_dropout`, the search range should be shifted down.
 - For `patch_projector_dropout`, search range should be shifted up.
- 6 trials pruned (4, 5, 6, 7, 8, 9).
 - Not a very meaningful run.
- 7 trials completed. Epoch of best validation loss:
 - Trial 0 → unclear
 - Trial 1 → before epoch 5
 - Trial 2 → epoch 5
 - Trial 3 → unclear
- Best trial: Trial 1
 - Best validation loss: 0.3177
 - Best validation accuracy: 0.7895

Hyperparameter	Range	Best value
<code>initial_learning_rate</code>	(1.8e-4 3.2e-4)	2.54e-04
<code>weight_decay</code>	(1.5e-4 3.8e-4)	2.90e-04
<code>patch_projector_dropout</code>	(0.0 0.3)	6.18e-03
<code>classifier_dropout</code>	(0.2 0.6)	0.5880
<code>class_weight_benign</code>	(2.5 3.5)	3.3324
<code>entropy_coefficient</code>	(2e-5 2e-4)	3.26e-05

Hyperparameter Tuning - Run 2 Results

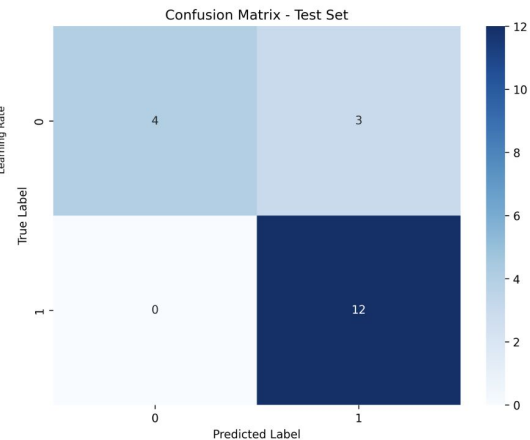
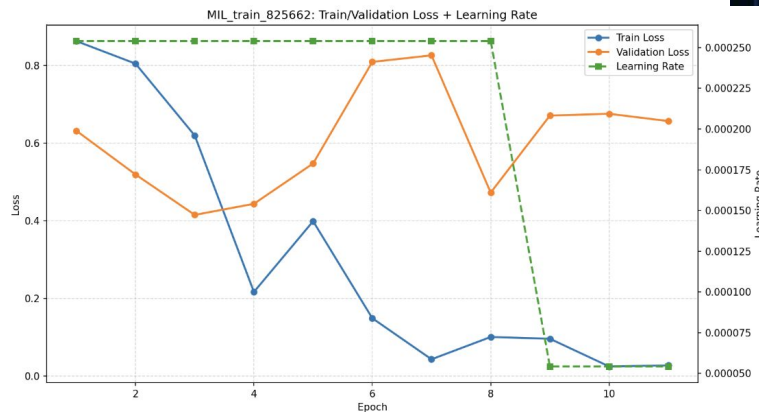
- Once again, best hyperparameters from Run 2 inserted into model for full training run.
- Same issue with memorization, jagged validation loss, model struggling to learn stable generalizable patterns.
- Classifier dropout remained high, leading to instability in predictions.
- Patch projector dropout decreased further, adding no regularization to early feature extraction layers.
- Benign class weight increased slightly, likely overshadowed by unstable training dynamics due to poor dropout values.
- Run 2 tuning did not improve performance as the narrowed search space continued to favor configurations that destabilized the classifier rather than improving generalization.

```
29 TRAINING_CONFIG = {  
30     'epochs': 30, # Increased since we have early stopping  
31     'batch_size': 1, # MIL typically uses batch_size=1  
32     'learning_rate': 2.54e-04, # Tuned value (run 2)  
33     'weight_decay': 2.90e-04, # Tuned value (run 2)  
34     'patch_projector_dropout': 6.18e-03, # Tuned value (run 2)  
35     'classifier_dropout': 0.5880, # Tuned value (run 2)  
36     'class_weights': [3.3324, 1.0], # Tuned benign class weight (run 2)  
37     'entropy_weight': 3.26e-05, # Tuned value (run 2)  
}
```

Test Results:

Test Loss: 0.7644

Test Acc: 0.8421



Hyperparameter Tuning - Optuna Run 3

- Run configuration: 10 trials, 30 epochs per trial, 6 epoch pruning warm up, **early stopping with patience 8**.
- Search the same ranges as in Run 2.
- 4 trials pruned (4, 5, 8, 9).
- 6 trials completed. Best validation loss; early stopping:
 - Trial 0 → best val loss epoch 4; early stopping epoch 12.
 - Trial 1 → best val loss epoch 2; early stopping epoch 10.
 - Trial 2 → best val loss epoch 2; early stopping epoch 10.
 - Trial 3 → best val loss epoch 3; early stopping epoch 11.
 - Trial 6 → best val loss epoch 4; early stopping epoch 12.
 - Trial 7 → best val loss epoch 3; early stopping epoch 11.
- Best trial: Trial 7
 - Best validation loss: 0.2926
 - Best validation accuracy: 0.8421

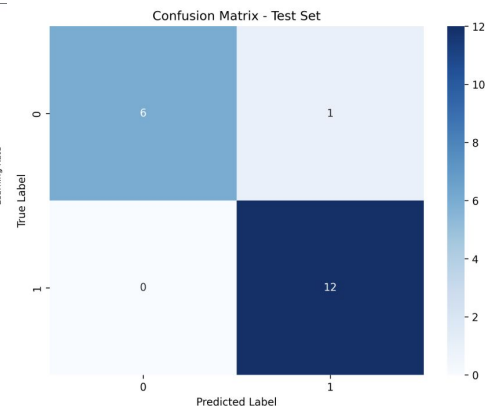
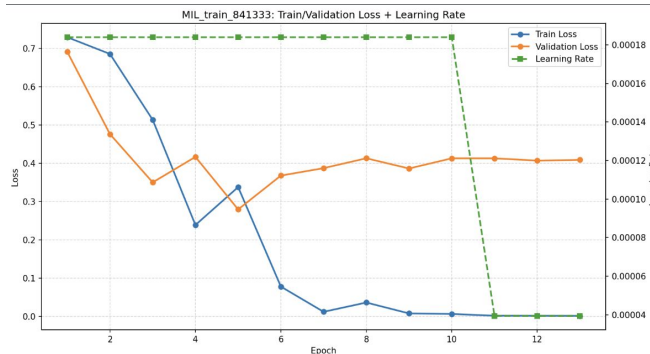
Hyperparameter	Range	Best value
initial_learning_rate	(1.8e-4 3.2e-4)	1.84e-04
weight_decay	(1.5e-4 3.8e-4)	3.49e-04
patch_projector_dropout	(0.0 0.3)	0.0776
classifier_dropout	(0.2 0.6)	0.4650
class_weight_benign	(2.5 3.5)	2.8117
entropy_coefficient	(2e-5 2e-4)	6.62e-05

Hyperparameter Tuning - Run 3 Results

- Best hyperparameters from Run 3 inserted, model trained fully.
- Introducing early stopping (patience =8) prevented unnecessary training epochs after validation loss stopped improving.
- Validation loss stabilizing earlier than previous runs → smoother convergence.
- The selected best hyperparameters improved model stability:
 - Classifier dropout = 0.465 → down from 0.58, allowing the classifier to learn stronger decision boundaries.
 - Patch projector dropout = 0.0776 → up from 0.006, reducing memorization of individual patches and improving generalization.
 - Benign class weight = 0.281, down from ~3.3, creating a more balanced decision boundary between benign and high-grade predictions.
- Overall less extreme dropout values and more balanced class weighting improve generalization – search more in these ranges for Presentation 4.

```
29 TRAINING_CONFIG = {  
30     'epochs': 30, # Increased since we have early stopping  
31     'batch_size': 1, # MIL typically uses batch_size=1  
32     'learning_rate': 1.84e-04, # Tuned value (run 3, w/ early stopping)  
33     'weight_decay': 3.49e-04, # Tuned value (run 3, w/ early stopping)  
34     'patch_projector_dropout': 0.0776, # Tuned value (run 3, w/ early stopping)  
35     'classifier_dropout': 0.4650, # Tuned value (run 3, w/ early stopping)  
36     'class_weights': [2.8117, 1.0], # Tuned benign class weight (run 3, w/ early stopping)  
37     'entropy_weight': 6.62e-05, # Tuned value (run 3, w/ early stopping)
```

Test Results:
Test Loss: 0.3283
Test Acc: 0.9474



Hyperparameter Tuning - Comparing Optuna Runs

- **Run 1** performed best, achieving the lowest val loss (0.1914) and highest val accuracy (0.8947) → broader hyperparameter search allowed Optuna to find better configurations.
- **Run 2** degraded performance after narrowing the search space → restricting parameter ranges limited Optuna's ability to explore better solutions.
- **Run 3** improved slightly over Run 2 with early stopping, which reduced unnecessary training but still did not outperform the broader search in the first run.

Across runs, training consistently showed signs of memorization, with validation loss fluctuations indicating difficulty learning stable, generalizable patterns.

<u>Feature</u>	<u>Optuna Run 1</u>	<u>Optuna Run 2</u>	<u>Optuna Run 3 (early stopping)</u>
Trials	10	10	10
Epochs	30	15	30
Search	Broad exploration	Narrowed search	Narrowed search
Completed trials	7	4	6
Pruned trials	3	6	4
Best validation loss	0.1914	0.3177	0.2926
Best validation accuracy	0.8947	0.7895	0.8421

Conclusion: Results indicate that broader hyperparameter exploration was more effective than narrowing ranges early, and search space design had a larger impact than training length.

/projects/e32998/STAT390_Winter2026/Team1/Code/Andrew_pres3_hyperparam_tuning/logs/
OPTUNA_Tuning_P3_456523.out
/projects/e32998/STAT390_Winter2026/Team1/Code/Andrew_pres3_hyperparam_tuning/logs/
OPTUNA_Tuning_P3_728608.out
/projects/e32998/STAT390_Winter2026/Team1/Code/Andrew_pres3_hyperparam_tuning/logs/
OPTUNA_Tuning_P3_790774.out

Hyperparameter Tuning - Full Run Results

- Across runs learning rates converged to similar range ($1.8\text{e-}4$ - $2.6\text{e-}4$) – this should be a stable region for the model.
- Extremely high classifier dropout in Runs 1 and 2 weakened the classifier and reduced performance, while the moderate value in Run 3 improved stability.
- Very small patch project dropout in Runs 1 and 2 reduced regularization and encouraged memorization, while the larger value in Run 3 improved stability.
- Run 3 achieved 94% test accuracy, matching they baseline while using a different hyperparameter configuration, indicating that balanced dropout values and early stopping improved generalization compared to earlier runs.

	<u>Baseline model (previous tuned hyperparameters)</u>	<u>Optuna run 1 best model</u>	<u>Optuna Run 2 best model</u>	<u>Optuna Run 3 best model</u>
initial_learning_rate	2.08e-4	2.63e-04	2.54e-04	1.84e-04
weight_decay	3.05e-4	2.55e-04	2.90e-04	3.49e-04
patch_projector_dropout	0.2599	0.0124	6.18e-03	0.0776
classifier_dropout	0.2599	0.5819	0.5880	0.4650
class_weight_benign	2.5285	3.1649	3.3324	2.8117
entropy_weight	0.1	7.07e-05	3.26e-05	6.62e-05
Test Accuracy:	94%	89%	84%	94%

Hyperparameter Tuning - Next Steps

- Run configurations:
 - Increase number of trials from 10 → 25-40.
 - Allow Optuna to explore the hyperparameter space more thoroughly.
 - Use 25-30 epochs per trial with early stopping maintained.
 - Reduce unnecessary epochs with early stopping instead of reducing epochs manually.
 - Potentially increase patience to 10 to ensure best validation losses are attained.
 - Increase pruning warm-up to 8-10 epochs.
 - Give the model more time to learn before abandoning trials.
- Hyperparameter search ranges:
 - One idea is to remove the `initial_learning_rate` and `weight_decay` from the hyperparameter tuning processes. These hyperparameters seem to have consistent values, it could be beneficial to focus on the dropouts, class weights, and entropy coefficient.
 - Adjust classifier dropout to lower range. Suggestion: (0.2, 0.45).
 - Adjust patch projector dropout to higher range. Suggestion: (0.1, 0.35).
 - Adjust benign class weight range. Suggestion: (1, 3).

Math - Sid – Task 1

Current Model: Patch $\rightarrow$ Slice $\rightarrow$ Stain $\rightarrow$ Case

At each level, we compute attention weights.

Patch Aggregation $\rightarrow$ Slice:
$$s_j = \sum_{p=1}^{P_j} \alpha_{jp} z_{jp}$$

z_{jp} = embedding of patch p in slice j

α_{jp} = patch attention weights

Slice Aggregation $\rightarrow$ Stain:
$$t = \sum_{j=1}^J \beta_j s_j$$

s_j = slice embedding

β_j = slice attention weights

Softmax Normalization:
$$\sum_p \alpha_{jp} = 1$$
$$\sum_j \beta_j = 1$$

Alpha & beta operate on different probability spaces. Serve different functions!

Math - Sid – Task 1

Claim:

“If patch attention remains highly peaky despite regularization, this indicates strong evidence of class membership. Therefore, slices containing peaky-attention patches should themselves be important and receive high slice attention. Hence, when patch attention is strongly peaked, slice attention becomes redundant.”

We refute this claim.

Peaky patch attention means the slice embedding is dominated by a sparse number of patches (patches with higher attention scores, α).

But, slice attention depends on: $\beta_j = \frac{e^{b_j}}{\sum_m e^{b_m}}$ $b_j = f(s_j)$

So, slice attention depends on relative slice scores, not patch sharpness.

Example: Slice 1 with $\alpha = [0.95, 0.05]$, Slice 2 with $\alpha = [0.90, 0.10]$

Both have peaky patch attention, but, regardless, if $f(s_2) > f(s_1)$ then $\beta_2 > \beta_1$.

Note that many patch distributions can produce the same slice embedding. Also, since we are doing entropy regularization, the slice is even less dependent on a single patch. Peaky $\alpha \nRightarrow$ large β

Math - Sid – Task 1

If we remove slice-level hierarchy, our new model is: Patch $\rightarrow$ Stain $\rightarrow$ Case

Instead of two-stage aggregation, aggregating from Patch $\rightarrow$ Slice $\rightarrow$ Stain. We aggregate directly from patch to stain.

$$t = \sum_{j,p} \tilde{\alpha}_{jp} z_{jp} \qquad \tilde{\alpha}_{jp} = \frac{e^{a_{jp}}}{\sum_{m,q} e^{a_{mq}}}$$

This removes slice-level competition.

—

The original hierarchy enforces:

1. Patch competition within slices
2. Slice competition across slices

Removing slice-level attention merges both competitions into one stage: Patch $\rightarrow$ Stain.

Math - Sid – Task 1

Should we drop slice attention?

Scenario I: Slice Attention Approximately Uniform $\beta_j \approx \frac{1}{J}$ $t \approx \frac{1}{J} \sum_j s_j$

In this scenario, slice attention essentially serves as average pooling.

So, removing slice attention in Scenario I will likely not harm performance and may simplify the model. We can remove it.

Scenario II: Slice Attention Highly Peaky $\beta_{j^*} \approx 1$ $t \approx s_{j^*}$

Slice attention is a coarse selector of relevant slices here. Removing this level may allow irrelevant slices to influence aggregation, weakening our signal. Thus, slice hierarchy is beneficial!

We should keep it.

Math - Sid – Task 1

Should we drop slice attention?

Scenario 3: Mixed Behavior Across Cases

Some cases:

$$\beta_j \approx \frac{1}{J}$$

Others:

$$\beta_{j^*} \approx 1$$

Slice-level attention adapts between diffuse (more uniform) evidence aggregation and sparse (more peaky) evidence selection. Removing the slice layer removes this adaptive capacity. Keep!

Final Recommendation (All Scenarios Accounted For): KEEP SLICE ATTENTION!

As of now, our model tends to have slightly more peaky behavior for slice attention. We do not want to lose the ability to aggregate this information (which is distinct from the patch level).

Math - Sid - Task 3

Resolution R determines patch count $N=R^2/P^2$ which propagates through every level of the hierarchy. N is number of patches, P is patch size in pixels. This is a structural decision that affects complexity, attention behavior, entropy regularization, and generalization bounds simultaneously.

We use gated attention now. At each level:

$$a_k = \frac{\exp(\mathbf{w}^\top (\tanh(\mathbf{V}\mathbf{h}_k) \odot \sigma(\mathbf{U}\mathbf{h}_k)))}{\sum_j \exp(\mathbf{w}^\top (\tanh(\mathbf{V}\mathbf{h}_j) \odot \sigma(\mathbf{U}\mathbf{h}_j)))}$$

The gate is $\sigma(\mathbf{U}\mathbf{h}_k)$ elementwise and there are no cross-patch interactions. Therefore complexity is $O(N)$ not $O(N^2)$.

The total complexity across the hierarchy is bottlenecked by the patch level as $N \gg N_s \gg N_{st}$. So, total complexity = $O(N)$.

Therefore, substituting $N = R^2/P^2$, $O(R^2/P^2)$ scales quadratically with resolution.

Memory is also $O(N \cdot d)$ where d is the embedding dimension as patch-level again is the bottleneck. High R dominates GPU budget.

$$O(N) + O(N_s) + O(N_{st}) \approx O(N) = O\left(\frac{R^2}{P^2}\right)$$

$$\underbrace{N = R^2/P^2}_{\text{patch level}} \implies \begin{cases} \text{compute} \propto R^2 \\ \text{memory} \propto R^2 \cdot d \end{cases}$$

Math - Sid - Task 3

We use entropy regularization now, encouraging diffuse attention.

Max entropy is achieved by uniform distribution:

This ceiling rises logarithmically with R.

Higher R raises Hmax, giving the entropy term more headroom.

At fixed lambda, higher R, silently increases regularization.

This risks the model over spreading attention and missing out on discriminative peaks.

Lambda and R are coupled. $\lambda_{\text{eff}} \propto \lambda / \log(R^2 / P^2)$

$$\mathcal{L}_{\text{reg}} = -\lambda \sum_k a_k \log a_k$$

$$H_{\text{max}} = \log N = \log \frac{R^2}{P^2}$$

Math - Sid - Task 3

Below P^2 Patches pixels – the lesion gets no patch. No model can use information that isn't captured in the input!

Resolution	20×20 lesion becomes	Detectable?
4096×4096	20×20 px	✓
1024×1024	5×5 px	Marginal
512×512	2×2 px	✗

Math - Sid - Task 3

More patches $\neq$ more supervision. Labels are at case level only.

$n_{\text{eff}} = n_{\text{cases}}$ regardless of R .

Higher R widens the generalization gap with no new labels.

Problematic patch attention propagates upward through all 4 levels.

More entropy regularization at high R makes this worse.

—

Note at high R , we have high bias and low variance from more diffuse attention. At Low R , we have the opposite effect.

$$\epsilon_{\text{gen}} \sim \sqrt{\frac{R^2 / P^2}{n_{\text{cases}}}}$$

Math - Sid - Task 3

If we stick with variable resolution, we need to tune R , λ , and P independently.

$$\lambda \propto \frac{1}{\log(R^2/P^2)} \quad \text{as } R \uparrow$$

$N=R^2/P^2$ sets entropy ceiling, generalization gap, and lesion sensitivity simultaneously. It is a structural hyperparameter.

We need to hardcap $R_{\max}$.

Grid: $R \in \{512, 1024, 2048\}$, $P \in \{8, 16, 32\}$, λ scaled accordingly

Evaluate: validation AUC, attention entropy, memory, convergence

Math - Sid - Task 3

What about fixed resolution?

Note in fixed:

$$\epsilon_{\text{gen}} \sim \sqrt{R^2 / P^2 \cdot n_{\text{cases}}}$$

CONSTANT!

	Fixed R	Variable R
H_{max} per image	✓ identical	✗ varies per case
λ consistency	✓ stable	✗ silently shifts
Small lesion sensitivity	✗ downsampling risk	✓ native resolution
Batch memory	✓ uniform	✗ needs padding
Generalization bound	✓ constant	✗ case-dependent
Scanner heterogeneity	✗ penalizes diversity	✓ handles naturally

If our dataset, is using multiple scanners and small lesion sensitivity is critical we should be using variable resolution. Otherwise, fixed resolution is the best path forward.

Thank
You!